

Tacet: An Honest Approach to Value Reification

Ramon Opazo

ramon.opazo.c@gmail.com

github.com/RamonOpazo/tacet-lang

Independent Research

August 19, 2026

Contents

1	Introduction	4
1.1	Models of failure placement	4
	<i>In the value type (monadic encapsulation)</i>	4
	<i>Nowhere in the type (exceptions)</i>	5
	<i>In the typing judgment (ours)</i>	5
1.2	Strong and weak value spaces	5
1.3	Honesty is disclosure, not the absence of failure	6
1.4	Contributions	7
1.5	Structure of the paper	7
2	Formal semantics	8
2.1	Function evaluation, the result domain, and reduction	8
	<i>Two-stage evaluation</i>	8
	<i>The result domain</i>	9
	<i>Reduction with fuel</i>	9
	<i>Boundedness and totality</i>	10
2.2	Dispositions and their composition	11
	<i>The disposition of an outcome</i>	11
	<i>Renunciation</i>	13
	<i>Non-termination without bottom</i>	13
	<i>Composition</i>	14
	<i>Discharge is uniform</i>	14
2.3	Witnesses and obligations	15
	<i>Thick witnesses, thin obligations</i>	15
	<i>Exposure</i>	15
	<i>Occurrences, not names: the Grothendieck reading of L_f</i>	16
	<i>The linear context, by inheritance</i>	16
	<i>The force of a dropped obligation</i>	17

Contents

2.4	Introduction	17
	<i>Symbols, and tacet as obligation introduction</i>	17
	<i>How each obligation arises</i>	18
	<i>Introduction rules</i>	18
	<i>The witness contract</i>	19
	<i>The correspondence $L_f \leftrightarrow \Delta$</i>	19
2.5	Discharge	20
	<i>The two contexts</i>	20
	<i>The three reifiers</i>	20
	<i>The type discipline of reification</i>	21
	<i>Deferral chains</i>	21
	<i>Additive branch combination</i>	22
	<i>Summary of linear-context transformations</i>	22
3	Metatheory	23
3.1	What we prove, and why it matters	23
3.2	The formal system, recalled	24
3.3	Structural lemmas	24
3.4	Syntactic soundness	25
3.5	Linear soundness	26
3.6	Honesty	26
	<i>A worked trace</i>	27
	<i>What honesty does not claim</i>	28

1 Introduction

Mainstream type systems provide weaker execution guarantees than their function types appear to suggest, and they leave that weakening implicit. A signature $f : A \rightarrow \tau$ claims that evaluation, given an input in A , produces a value of type τ . Yet in most languages evaluation may instead terminate through a panic, propagate an exception, or fail to terminate at all — and none of these possibilities is visible in the type. The declared type and the semantic contract it seems to establish diverge, silently.

The divergence can be made precise. Consider the *semantic outcome space* of a function,

$$\llbracket f \rrbracket : A \longrightarrow \tau + E + \{\perp\},$$

where τ ranges over successful values, E over exceptional termination, and $\perp$ over computations that yield no value at all, such as divergence or undefined evaluation. A mainstream type system typically exposes only

$$f : A \longrightarrow \tau,$$

suppressing E and $\perp$ entirely. The critical dishonesty is that the type describes only the successful, value-producing behaviour of a function, while the signature is read as though it described the whole of $\llbracket f \rrbracket$. The gap between the two is the subject of this work. We seek a calculus that keeps the declared type honest with respect to the semantic outcome space, while exposing only the minimal machinery required to close that gap.

1.1 Models of failure placement

Existing disciplines for handling failure differ, in essence, over a single question: *where is failure reified?* Three answers are already in wide use, and the contrast between them motivates ours.

In the value type (monadic encapsulation)

Languages such as Haskell may represent failure explicitly in the return type, transforming a function that might fail from $A \rightarrow B$ into $A \rightarrow \text{Maybe } B$. The

possibility of failure is thereby made static and propagated through subsequent computation. But the failure has been relocated *into the value domain*: Maybe B replaces the domain B with one that additionally represents absence, so the caller must continuously reason about the computational context — the wrapper — alongside the underlying value. The resulting function is total with respect to its monadic domain, yet its value space has been weakened, because the absence of a value is now representable within the enclosing type.

Nowhere in the type (exceptions)

Languages such as Python model failure as a runtime control-flow object that interrupts evaluation and propagates outward until handled. A function annotated $f : A \rightarrow B$ may return a value of type B , or it may raise, with no indication of that possibility in its type. The value space stays clean — a returned B really is a B — but honesty is lost: the signature asserts a totality the function does not possess, and the exceptional path remains an invisible control-flow escape.

These two disciplines sit at opposite corners of a trade-off. The monadic discipline buys honesty by weakening the value space; the exceptional discipline preserves the value space by abandoning honesty.

In the typing judgment (ours)

We take a third position. Failure is reified neither inside the value type nor nowhere, but in the *typing judgment itself*, as a linear obligation recorded alongside — not inside — the value type. Judgments take the form

$$\Gamma; \Delta \vdash e : \tau,$$

where τ is the ordinary value type, kept clean, and Δ is a linear context of obligations the computation must discharge. The value type continues to promise exactly τ ; the honesty is carried by Δ . This is the central move of the paper: we obtain a signature that is honest about failure *without* enriching τ , by placing the obligation in the context rather than in the value.

1.2 Strong and weak value spaces

The distinction just drawn can be stated once and for all. A value space is **weak** when computational failure is representable within the same domain as successful values: Maybe B is weak because the absence of a B inhabits the very type that is supposed to contain values of B . A value space is **strong** when it preserves the separation between values and computational outcomes: τ contains only values of type τ , while failure, divergence, and exceptional termination remain distinct computational outcomes, tracked elsewhere.

Tacet maintains a strong value space. No element of τ ever denotes a failure, and $\perp$ is never an inhabitant of a value type. A strong value space has no bottom, and therefore no representable *absence*: Tacet has no null, no none, no nothing, no falsy stand-in for a missing value. The honest way to say “there is no value here” is not a degenerate inhabitant but a reified obligation — so absence is a computational outcome, never a value. Whatever can go wrong in a computation is accounted for outside τ — and, as the next section explains, accounted for

explicitly.

1.3 Honesty is disclosure, not the absence of failure

We do not attempt to abolish failure, and we cannot abolish divergence — the halting problem forbids deciding, in general, whether an arbitrary computation terminates. Our principle is weaker and attainable: every way in which a computation may deviate from returning a value of its declared type must be **disclosed**. Honesty, in Tacet, is not the absence of $\perp$ but the disclosure of its possibility.

The mechanism rests on a three-level separation between **values**, **witnesses**, and **obligations**.

Ordinary evaluation produces a *value* — an element of the value domain V . When evaluation instead encounters a computational phenomenon that is not an ordinary value — a possible failure, an unbounded recursion, an asynchronous wait, an intrinsically infinite loop — it raises an *obligation*: a thin, nominal signal whose entire content is its own identity, marking that a value is owed here and must be restored. Each obligation has a thick companion, its *witness* — not a value but the *type* that records what is owed, including the type of the value that was expected. The obligation is what the computation tracks and must discharge; the witness is what a function’s interface advertises about it.

The tacet construct is the syntactic bridge that lets a computation raise such an obligation deliberately. The name is the musical direction *tacet* — “it is silent,” the instruction that an instrument plays nothing for a passage: where a value would sound, tacet leaves a rest. It silences a position where a value would sit, leaving an obligation in its place, and reify is what breaks the silence, restoring a value to the value space. Because an obligation is not a value, tacet names it with a value-level *symbol* — a proxy that carries the obligation’s meaning while the obligation itself stays outside the value space. Once an obligation is exposed, it must be *discharged*, and discharge is always *reification*. Usually it reifies the obligation back into an ordinary value; it may instead reify not into a value but into a *new* obligation — deferring to a later step the decision of how to answer it, without ever leaving the discipline. Linearity governs the whole process: an obligation cannot be duplicated and cannot be silently dropped. Its only alternative to being discharged is an explicit, terminal failure — surfaced loudly, never absorbed into a quiet default or a silent bottom.

Some phenomena resist discharge by their nature. An unbounded recursion may never reach a value to recover, and a fire-and-forget effect has, by design, no value to recover. Such phenomena may be *renounced* rather than discharged — but only by declaring a corresponding **capability** in the function’s signature. Declaring the capability to run unboundedly, for instance, discloses to every caller that evaluation may not terminate; it does not thereby introduce a bottom value. Where a renounced computation does produce a result, that result is an ordinary value; where it does not — as with genuine non-termination — the outcome is simply the disclosed risk the caller was warned of, never a silent $\perp$. A function that declares no such capability is guaranteed, by construction, to return a value of its declared type. In this way the outcome space $\tau + E + \{\perp\}$ is honestly resolved: exceptional outcomes are reified and discharged, non-termination appears only where a capability announces it, and no bottom value is ever produced. Whatever a function can do beyond returning τ is precisely what its type and its capabilities

disclose.

1.4 Contributions

This paper develops the calculus sketched above and establishes its basic metatheory. Concretely, it contributes:

- a computational model that separates values from obligations, sustaining a **strong value space under honest signatures** with minimal syntactic surface;
- a uniform treatment in which failure, divergence, asynchrony, and looping are all **witnessed computational outcomes** exposing linear obligations, organized by a small algebra of admissible *dispositions* whose structure constrains which obligations may co-occur;
- **linear discharge rules** — reification in several forms, one of which defers by reifying into a fresh obligation rather than a value — together with a type-preserving/type-transforming distinction that renders a branch’s codomain discipline statically analyzable; and
- a **metatheory**: preservation and progress for the linear fragment, a conservation result stating that no obligation is ever silently dropped, and an **honesty theorem** establishing that the outcomes of a well-typed program coincide with its declared type, up to the capabilities and branch dispositions its signature explicitly declares.

1.5 Structure of the paper

Chapter 2 develops the formal semantics: it defines the result domain and the reduction relation, characterizes the disposition algebra that governs how witnesses compose, and gives the introduction and discharge rules through which obligations enter and leave the linear context. Chapter 3 establishes the metatheory — preservation, progress, conservation, and the honesty theorem that makes the central claim precise. Chapter 4 turns to the surface language, giving the grammar of Tacet and mapping each semantic rule of Chapters 2–3 onto the concrete syntax that triggers it.

2 Formal semantics

We now develop the calculus sketched in the introduction. Its organizing idea is a separation between *values* and *computational obligations*: the value a computation produces is kept distinct from the obligations its evaluation incurs, so that a strong value space can be maintained under an honest signature.

The chapter proceeds in five steps. Section 2.1 fixes the result domain into which evaluation lands, together with the small-step reduction relation — equipped with a fuel discipline — on which the metatheory of Chapter 3 depends. Section 2.2 introduces *dispositions*, the small algebra that governs which computational phenomena may co-occur. Section 2.3 characterizes witnesses and the obligations they expose, distinguishing the stateful witness from the nominal obligation. Section 2.4 gives the introduction forms through which obligations enter the linear context, and connects the semantic account of obligations to their syntactic bookkeeping. Section 2.5 gives the discharge rules through which obligations are consumed. The metatheory these definitions support — culminating in the honesty theorem — is developed separately in Chapter 3.

2.1 Function evaluation, the result domain, and reduction

Two-stage evaluation

Given a function f , let A denote its input-value domain, B the universe of branches, and $B_f \subseteq B$ the set of branches occurring in the body of f . Evaluation proceeds in two stages: an input first determines the branch to be executed, and that branch is then evaluated,

$$A \xrightarrow{\text{select}_f} B_f \xrightarrow{\text{eval}} R.$$

The selection stage captures the control-flow structure of f : for an input $a \in A$, select_f identifies the branch $b \in B_f$ whose execution a determines. Concretely, select_f is the language’s match construct, written $\triangleright$, which dispatches over the arms of f by matching the scrutinee along whichever axis its arms use — *truthiness*, *structure*, *content*, or *morphology*. Binary decision is the degenerate case rather than a separate primitive: Tacet has no boolean atom, only Church booleans, the total selectors $\text{true} = x\ y \mapsto x$ and $\text{false} = x\ y \mapsto y$, and a two-arm

truthiness match is exactly such a selector applied to its arms.

Selection is *total and obligation-free by construction*. Matching only chooses an arm; it never itself raises, and — because a Church boolean is a total value in V — the decision axis lives wholly inside the value space. There is no implicit “non-exhaustive match” failure: if no arm should produce a value, the programmer writes that arm explicitly as *tacet*, so even the gap in a match is a disclosed obligation rather than a silent error. Every obligation in the system therefore enters through introduction (Section 2.4), never through branch selection. The evaluation stage then reduces the chosen branch b to an element of the *result domain* R .

The inclusion of B_f into the general branch universe,

$$B_f \hookrightarrow B,$$

lets the semantics reason about the branches of a particular function without identifying them with the entire syntactic universe of possible branches. The structure of B_f determines which branch-specific phenomena can arise during the evaluation of f .

The result domain

The result domain is partitioned into values and witnesses,

$$R = V \uplus W.$$

Here V is the domain of ordinary language values and W is the domain of witnessed outcomes — obligation-bearing results, each carrying a witness type. The disjointness is deliberate: a witnessed outcome is not a language value and cannot be used interchangeably with an element of V . Ordinary evaluation has the form $b \mapsto v \in V$, while a branch that leaves an obligation has the form $b \mapsto w \in W$. Thus R is the complete range of branch-evaluation outcomes, and V is the sub-range consisting of ordinary values.

The value domain is moreover kept apart from the bottom element,

$$\perp \notin V.$$

Bottom is not an ordinary value and is never produced as the outcome of a successful evaluation. In particular there is no null, none, or nothing inhabiting any value type: absence is never a value but a Reify obligation, reached only through *tacet* (Section 2.4). This matters because the semantics does not model the phenomena carried by witnesses as ordinary bottom-valued computation; such phenomena are represented explicitly, as elements of W (Section 2.3). We will shortly strengthen $\perp \notin V$ to $\perp \notin R$, which holds unconditionally, in every evaluation regime.

Reduction with fuel

To state progress, preservation, and — above all — honesty, we need a reduction relation to reason about, not merely the input–output map *eval*. We equip evaluation with a *fuel* budget that bounds the depth of self-referential computation.

A **configuration** is a pair $\langle e, \varphi \rangle$ consisting of an expression e and a fuel value

$$\varphi \in \mathbb{N} \cup \{\infty\}.$$

Reduction is a relation $\langle e, \varphi \rangle \rightarrow \langle e', \varphi' \rangle$ on configurations, governed by two disciplines.

(Fuel.) The two reduction forms that can recur without structurally shrinking the expression — the unfolding of a recursive call and the deferral reifier reify_L (Section 2.5) — each draw one unit of fuel:

$$\frac{\varphi > 0}{\langle \mathcal{R}[e], \varphi \rangle \rightarrow \langle \mathcal{R}'[e], \varphi - 1 \rangle} \quad (\text{recursion / defer}),$$

with the convention $\infty - 1 = \infty$. When such a redex is reached at $\varphi = 0$, it does not become stuck and does not diverge; it introduces a Bound obligation — the obligation of an Unbounded witness (Section 2.2) — recording that the budget was exhausted:

$$\overline{\langle \mathcal{R}[e], 0 \rangle \rightarrow \langle \text{obl}_{\text{Bound}}, 0 \rangle}.$$

Every other reduction step leaves the fuel value unchanged.

(Capability.) A function evaluates with $\varphi = \infty$ if and only if it declares the Unbounded_0 capability in its signature; otherwise it evaluates with a finite budget $\varphi \in \mathbb{N}$. Thus the two regimes are one relation at two fuel settings, and the capability reads semantically as the lifting of the fuel bound. Because Unbounded_0 is recorded in the signature, the choice of regime is disclosed to every caller.

Boundedness and totality

The result domain never contains bottom:

$$\perp \notin R,$$

unconditionally, in either fuel regime. Tacet does not model non-termination as a value; what the two regimes differ in is only whether reduction is *guaranteed to halt*.

In the default regime, $\varphi \in \mathbb{N}$ is finite. Because recursion and deferral strictly decrease φ and every other step leaves it fixed, no configuration admits an infinite reduction sequence: fuel exhaustion converts would-be divergence into a finite reduction ending in a Bound obligation. Evaluation therefore always halts, eval is *total* on B_f , and every branch reduces to some element of R . The fuel-exhaustion outcome is not $\perp$; it is an ordinary Bound obligation, typed as an Unbounded witness in W . Like every obligation, it is discharged through the single reification API (Section 2.5); its *default resolution*, Bound , is precisely the fuel discipline just described. Section 2.2 fixes these per-kind default resolutions, and Section 2.5 the uniform reifiers that can override them.

In the renounced regime, $\varphi = \infty$, the fuel argument no longer forces termination, and reduction of such a branch may continue without end. This does *not* reintroduce $\perp$. The possibility of non-termination is disclosed by the Unbounded_0 capability in the signature; the branch is recorded by a null-obligation witness in W_0 — bookkeeping that owes nothing and ultimately maps to Total (Section 2.2). A computation that fails to halt produces no semantic value at all, but it produces no bottom either: $\perp$ remains outside R , and the risk it runs is exactly the one its

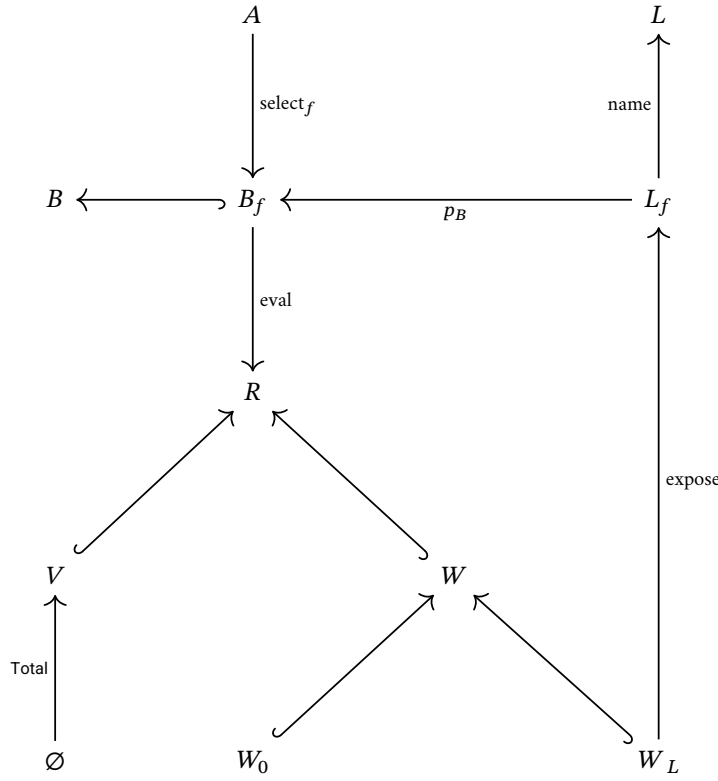


Figure 2.1: Semantic structure of function evaluation. An input selects a branch, $\text{select}_f : A \rightarrow B_f$, which evaluates, $\text{eval} : B_f \rightarrow R$, into the result domain $R = V \uplus W$ with $W = W_0 \uplus W_L$ (the hooked arrows are the coproduct injections). The three regions are the fibres of the disposition: V holds Total outcomes — ordinary values, into which the empty witnessing type maps via $\text{Total} : \emptyset \rightarrow V$ — while W_L holds witnesses that expose a linear obligation and W_0 holds null-obligation witnesses, renounced book-keeping that owes nothing and ultimately maps to Total. Bottom is not an object of the diagram: $\perp \notin R$, unconditionally. On the obligation side, L_f is the total space of the family obligations $: B_f \rightarrow \mathbf{Set}$ (Section 2.3), carrying the projection $p_B : L_f \rightarrow B_f$ and the generally non-injective name $: L_f \rightarrow L$; witnesses meet obligations through the set-valued $\text{expose} : W_L \rightarrow \mathcal{P}_{\text{fin}}(L_f)$. The internal structure of V , W , and L_f is developed in Sections 2.2–2.3.

capability announced. This is the governing principle in its sharpest form: honesty is the *disclosure of the risk*, never the production of $\perp$.

The overall shape of function evaluation is summarized in Figure 2.1; the internal structure of V and W indicated there is developed in the sections that follow.

2.2 Dispositions and their composition

The disposition of an outcome

The partition of the result domain in Section 2.1 is not primitive; it is induced by the *disposition* of a branch evaluation — the collection of computational phenom-

ena its evaluation realizes. The default disposition is *Total*, which is not a witness kind but the name of the *empty* witnessing type: it realizes no phenomenon and exposes nothing, and for exactly that reason it maps to an ordinary value in V . The Church booleans $\text{true} = x\ y \mapsto x$ and $\text{false} = x\ y \mapsto y$ are ordinary such values — total functions that select between two values already held, raising nothing — and it is the empty witness *Total* that places them in V . The *decision* axis of the language thus lives wholly in V , disjoint from the *absence* axis, which lives wholly in L and is reached only through *tacet*; the two are never conflated in a single degenerate inhabitant.

When evaluation realizes a phenomenon that is not an ordinary value — a possible failure, an unbounded recursion, an asynchronous wait, an intrinsically infinite loop — the branch acquires a corresponding *witness kind*. The linear kinds are

$$\mathcal{K}_L = \{\text{Partial}, \text{Unbounded}, \text{Awaited}, \text{Loop}\}.$$

This set is closed and exhaustive. Its four kinds are the orthogonal axes along which a computation can depart from producing a value — *partiality*, *boundedness*, *effectfulness*, and *loop productivity* — and no further axis inhabits the space. The metatheory of Chapter 3 is defined over exactly these four.

Each witness exposes a named obligation, or none; the obligation it exposes determines the region of the result domain it inhabits:

Witness	Exposes	Domain
Total ($\emptyset$)	-	V
Partial	Reify	W_L
Unbounded	Bound	W_L
Awaited	Sync	W_L
Loop	Break	W_L
Unbounded ₀	-	W_0
Awaited ₀	-	W_0

The four linear kinds each expose an obligation and inhabit W_L . The obligation's name is its entire content: consistent with the thin-obligation ontology of Section 2.3, a *Bound* obligation records nothing but the nominal fact that an unboundedness occurred and must be answered; a *Reify* obligation, that a partiality occurred; and so on. The minus-marked witnesses *Unbounded₀* and *Awaited₀* are the *renounced* forms, defined in the next subsection; they expose no obligation and inhabit W_0 . *Total*, the empty witnessing type, also exposes nothing — but unlike a W_0 witness it records no phenomenon at all, and so maps to V rather than to W .

The partition of Section 2.1 now reads off the disposition, into three regions:

$$\emptyset \rightarrow V, \quad \{\text{Partial}, \text{Unbounded}, \text{Awaited}, \text{Loop}\} \rightarrow W_L, \quad \{\text{Unbounded}_0, \text{Awaited}_0\} \rightarrow W_0.$$

A *Total* branch — the empty witness — lands in V ; a branch exposing at least one obligation lands in W_L ; a branch that is *witnessed but exposes no obligation*, renounced bookkeeping, lands in W_0 . So $R = V \uplus W$ with $W = W_0 \uplus W_L$ — three genuine regions, not two.

Renunciation

Two of the linear kinds admit a *renounced* form, written with a superscript minus — the witness with its obligation removed:

$$\text{Unbounded}_0 \quad \text{Awaited}_0$$

Renunciation removes the *obligation* from a witness while retaining the witness itself. The superscript minus is category-theoretic notation: κ_0 names the non-obliged counterpart of the witness κ . An Unbounded_0 or Awaited_0 witness inhabits W_0 : it is genuine bookkeeping — a record that the computation passed through a renounced-unbounded or fire-and-forget point — yet it exposes no obligation, by explicit declaration, and so imposes nothing to discharge. The phenomenon is not answered but *accepted*, and the acceptance is disclosed as a capability in the function’s signature (Section 2.1). A W_0 witness carries no obligation and ultimately maps to *Total*, hence into V , at the end of a computation (Section 3.6) — but it is a witness in W throughout, never itself a value.

Because a renounced witness exposes no obligation, it is inert in the composition algebra, which constrains only *live* linear kinds. A computation that is both failable and renounced-unbounded has live linear face $\{\text{Partial}\}$; the renounced axis contributes no obligation, so no admissibility question arises. This is why the exclusions of the composition algebra (below) constrain only the pairs of live linear kinds — $\{\text{Unbounded}, \text{Awaited}\}$ and the like — and never a renounced form.

Renunciation is available only for *Unbounded* and *Awaited* — the phenomena whose obligation one may legitimately decline: an unbounded recursion may be allowed to run, and an asynchronous effect may be launched without awaiting it (the fire-and-forget sink). Partiality and looping have no renounced form. A failure must be answered and a productive loop must be broken, so *Reify* and *Break* obligations cannot be declined — only discharged.

Non-termination without bottom

Three kinds bear on non-termination, and all three obey the principle that *Tacet* produces no bottom (Section 2.1); they differ only in how the possibility of non-termination is accounted for.

- *Unbounded* is fuel-bounded by default. Evaluation is guaranteed to halt; on budget exhaustion it introduces its *Bound* obligation. No unbounded run occurs.
- Unbounded_0 renounces the bound. Evaluation may run without end, disclosed by the capability; it is recorded as a W_0 witness — null-obligation bookkeeping, ultimately mapping to *Total* — and non-termination is never a bottom value.
- *Loop* exposes a *Break* obligation. A loop is intrinsically infinite: it does not draw on the fuel budget (a fuel-limited loop could not run forever, which would defeat the purpose of a productive one), and it iterates productively until its *Break* obligation is discharged. Uniquely among the kinds, a *Break* is discharged *out-of-band* — by an explicit break originating outside the loop body, either an interrupt (*Ctrl-C*) or a call to a breaking function — rather than within the forward flow of evaluation. Until that break lands

the branch does not complete; a loop never broken runs productively forever. This is not bottom: the branch simply never yields a value, and the Break obligation discloses, statically, that completion is contingent on an explicit break.

In every case the possibility of non-termination is reified — as a fuel bound, a renounced capability, or a Break obligation — and never as an element of R .

Composition

Dispositions compose. A single computation may realize several phenomena at once — an asynchronous computation that can also fail, say — so its disposition is in general a *set* of linear kinds. Composition is orthogonal: writing a branch's disposition as $S \subseteq \mathcal{K}_L$, the disposition algebra is

$$(\mathcal{D}, \otimes, \text{Total}), \quad \text{Total} = \emptyset, \quad S \otimes S' = S \cup S',$$

a commutative, associative, idempotent operation with Total as identity — a bounded join-semilattice — made *partial* by the fact that not every combination is admissible.

The inadmissible combinations are exactly the four pairs

$$\{\text{Loop}, \text{Unbounded}\}, \quad \{\text{Loop}, \text{Awaited}\}, \quad \{\text{Loop}, \text{Partial}\}, \quad \{\text{Unbounded}, \text{Awaited}\}.$$

Admissibility is determined entirely by pairs and is downward closed, so the admissible dispositions are exactly the *cliques* of the compatibility graph G whose edges are the two surviving pairs, Partial–Unbounded and Partial–Awaited. Thus G is a two-leaf star centred on Partial, together with an isolated Loop. Because inadmissibility lives entirely in pairs, the admissible dispositions form a **flag (clique) complex**, with seven faces,

$$\emptyset (\text{Total}), \quad \{\text{Partial}\}, \{\text{Unbounded}\}, \{\text{Awaited}\}, \{\text{Loop}\}, \quad \{\text{Partial}, \text{Unbounded}\}, \{\text{Partial}, \text{Awaited}\},$$

whose three maximal faces are the only maximal effect-profiles a computation may inhabit:

$$\underbrace{\{\text{Partial}, \text{Unbounded}\}}_{\text{failable recursion}} \quad \underbrace{\{\text{Partial}, \text{Awaited}\}}_{\text{failable async}} \quad \underbrace{\{\text{Loop}\}}_{\text{productive loop}}$$

Every non-Total computation is thus a face of one of three maximal simplices. Partial (failability) is the universal companion, pairing with either Unbounded or Awaited but never both; Loop is sealed, composing with nothing, because a productive loop answers its own failures internally rather than exposing them upward.

Discharge is uniform

Whatever obligation a W_L witness exposes — Reify, Bound, Sync, or Break — it is discharged through the single reification interface of Section 2.5: the three reifiers reify_T , reify_V , reify_L . There is no per-kind discharge machinery to tabulate. The name of an obligation records where it came from; the reifiers say, uniformly,

how it may leave — into a value of the same type, a value of another type, or a fresh obligation. This uniformity is precisely what allows one small interface to answer partiality of every kind.

2.3 Witnesses and obligations

Sections 2.1 and 2.2 have spoken of witnesses and of the obligations they expose as though these were the same kind of thing seen twice. They are not. A witness and an obligation stand in a definite ontological asymmetry, and making it explicit is what lets the linear bookkeeping of Section 2.5 sit consistently atop the semantic account given here.

Thick witnesses, thin obligations

A **witness** is *thick*. It is a *type*: the type carried by an obligation-bearing occurrence, recording everything statically known about it. It carries two things — the payload type T of the value that was expected, and the disposition $S \subseteq \mathcal{K}_L$, the face recording which phenomena the occurrence realizes — so its type is indexed by both,

$$\text{Witness}_S(T),$$

and is fixed at the point where the obligation is introduced, because the state that determines it is present there. A witness type knows what it is. It is what appears in a function’s contract, at a call boundary, and as the annotation on an occurrence in Δ ; it is never a separate runtime object floating alongside the value.

An **obligation** is *thin*. It carries no payload and no state; its entire content is its own identity. An obligation is a *signal* — a nominal token whose message is simply that a computation has left the value space at a definite point and must be brought back. The obligation exposed by a Partial witness is a Reify obligation, that exposed by an Unbounded witness a Bound obligation, and so on; the name is the meaning. Nothing about the recovered value’s type lives in the obligation — that lives in the witness.

The two are bridged by exposure, and the direction of the asymmetry is exactly that a thick witness gives rise to thin obligations, never the reverse. A witness whose disposition is empty — a null-obligation W_0 witness (Section 2.2) — is thick all the same, carrying its payload type as bookkeeping, but gives rise to no obligation at all; exposure below concerns the W_L witnesses, whose disposition is non-empty.

Exposure

A witness exposes one obligation occurrence per active axis of its disposition:

$$\text{expose} : W_L \longrightarrow \mathcal{P}_{\text{fin}}(L_f), \quad \text{expose}(w) = \{l_\kappa : \kappa \in S(w)\}.$$

For a witness of singleton disposition — say $S = \{\text{Partial}\}$ — this is a single Reify occurrence. For a composed witness — the only non-trivial admissible cases being $\{\text{Partial}, \text{Unbounded}\}$ and $\{\text{Partial}, \text{Awaited}\}$ (Section 2.2) — it is two occurrences at once, one per axis, each of which must be discharged independently. Exposure

is thus in general one-to-many; that a single witness may raise several obligations is precisely why the codomain is a finite powerset of L_f rather than L_f itself.

Occurrences, not names: the Grothendieck reading of L_f

The obligations of a function are collected across its branches. For each branch $b \in B_f$, let $\text{obligations}(b)$ denote the occurrences arising in b — those tacet introduced directly, together with those exposed by any witness types received at a call within b . The obligations of f are then the disjoint union

$$L_f = \coprod_{b \in B_f} \text{obligations}(b).$$

The disjoint union is doing real work: L_f tracks *occurrences*, not the distinct obligation names that occur. This is the total space of the family obligations : $B_f \rightarrow \mathbf{Set}$ — its Grothendieck construction f obligations — whose elements are pairs (b, l) of a branch and an occurrence within it. It comes with two canonical maps:

$$B_f \xleftarrow{p_B} L_f \xrightarrow{\text{name}} L,$$

the projection p_B recording *which branch* an occurrence arose in — its fibre over a branch recovers that branch's obligations, $p_B^{-1}(b) = \text{obligations}(b)$ — and the naming map name recording *which* underlying obligation it is an occurrence of, within the universe L of all obligations.

The point of the construction is that name need not be injective. If two distinct branches $b_1, b_2 \in B_f$ each raise an occurrence of the same underlying obligation $l \in L$, those occurrences are nevertheless distinct elements of L_f — distinguished by their fibres under p_B — even though name identifies them in L . Branch structure is thereby preserved when the obligations of a function are collected, and this is exactly why L_f is associated with the function while L is the universe: L describes the possible obligation names, whereas L_f records the particular occurrences the branches of f induce. (The original presentation wrote $L_f \hookrightarrow L$; the inclusion is too strong, since it would force distinct occurrences of one name to collapse. The correct arrow is the generally non-injective $\text{name} : L_f \rightarrow L$.)

The linear context, by inheritance

The bridge to the syntactic bookkeeping of Section 2.5 is now short. The linear context Δ tracks occurrences of L_f , and an entry $l : \text{Witness}(T)$ in Δ does not assert that the obligation *is* a witness. It records the occurrence l together with the payload type T *inherited from the witness that exposed it* — so that a reifier knows the type of the value its handler will receive. The obligation contributes its identity through name ; the witness contributes the type through expose . There is no tension between the semantic separation of W_L from L_f and the syntactic annotation of occurrences with witness types: the annotation is simply the image of a witness under expose , read together with the witness's own payload.

The force of a dropped obligation

An obligation’s message is: *rectify here, or produce no value*. In Tacet the second alternative is not a silent $\perp$ but an explicit, terminal, loudly surfaced failure. This is the semantic force behind treating obligations linearly. An unconsumed obligation cannot decay into a quiet default, because its only alternative to being discharged is a visible error — never a bottom, and never an implicit fallback. The linear discipline of Section 2.5, and the Global Branch Totality it enforces, is precisely the static guarantee that a well-typed branch never reaches that alternative: every occurrence in L_f that a branch raises is, in well-typed code, discharged before the branch completes.

2.4 Introduction

Section 2.3 characterized obligations and the domain L_f that collects them, but not how an obligation comes to be there in the first place. This section gives the *introduction* forms — the constructs through which a witness is raised and its obligations enter the linear context — and connects the semantic collection L_f to the syntactic context Δ that the discharge rules of Section 2.5 will consume.

Symbols, and tacet as obligation introduction

Obligations are not values: they live in L , outside the value space entirely, and so cannot be named, passed, or matched on directly in code. To let a program *refer* to an obligation without ever *holding* one, the language provides value-level proxies. A **symbol** is such a proxy — a value that stands in for an obligation as its avatar, carrying only the obligation’s semantic meaning and none of its linear substance. A **symbol-construct** is a set of symbols. Symbols are what code manipulates; the obligations they name stay in L .

A symbol-construct is a signal to the type interpreter, which reads it as a type-level operation against Γ or Δ : a check on values, a declaration of witnesses, or the production of an obligation. Symbol-constructs serve several roles in the language; module loading and creation (Load, Module) are introduced in a later chapter. Here we are concerned with the two that bear on obligations: the *tacet* keyword, which *introduces* obligations, and the function contract, which *declares* witnesses.

The *tacet* keyword is the most universal of the obligation-introducing rules, because it is simply the introduction written directly in the code. Applied to a symbol-construct, *tacet* signals that the branch produces the linear obligation the symbols name, and introduces that obligation into Δ . It can introduce *any* obligation the language admits, with a single exception: it cannot introduce **Break**. A **Break** is never raised by code — it is produced solely by a decision the user makes while the computation runs, an interrupt or a call to a breaking function — so no symbol introduces it. Every other obligation is within *tacet*’s reach.

The essential discipline is that *tacet* lands **purely in L** . It consumes a symbol — a value — and emits an obligation, which is not a value; the symbol is the avatar that carries the semantic meaning across this boundary, leaving the obligation itself wholly outside the value world. *tacet* never produces a witness. What it does do, beyond depositing the obligation in Δ , is *force an implicit witness in the contract*: a branch that raises an obligation via *tacet* obliges the function’s con-

tract to carry the corresponding witness (below). Witnesses (W) thus remain the concern of the interface, obligations (L) the concern of the body, and expose (Section 2.3) is the map between them at the call boundary — the two domains never merge, which is exactly why they are kept separate. Writing the elaboration with a squiggle, to mark that it is not an ordinary semantic morphism,

$$\text{tacet} : \text{Sym} \rightsquigarrow L_f,$$

we cross the value/obligation boundary exactly once, and nothing of L leaks back into the value space.

How each obligation arises

Each of the four obligations reaches the type system by its own route, and the routes are deliberately not uniform.

- Reify enters through tacet and through reify_L (Section 2.5): tacet raises it where a computation declares it may fail to produce a value, and reify_L threads it forward as the Partial successor of a deferral.
- Bound enters through tacet and through recursion under the fuel discipline (Section 2.1): budget exhaustion produces it, and a recursive computation that may not terminate carries it.
- Sync is a deliberate special case. It should be *completely transparent* to the user — Tacet does not want two colours of function, one synchronous and one not. Asynchrony is therefore expressed only by calling the language’s primitive effectful functions, which are lifted into progressively richer effectful ones; the introduction rule for Sync is a function, not a keyword, and an asynchronous function is syntactically indistinguishable from a synchronous one.
- Break cannot be introduced in code at all. It is produced only by the user’s runtime decision, so the Loop phenomenon it belongs to reaches the type system solely through the contract, as the next subsection explains.

Introduction rules

An introduction deposits a fresh obligation occurrence into the linear context, guarded by admissibility: the new kind must be compatible with those already live. Writing $\text{kinds}(\Delta)$ for the witness kinds live in Δ and $\mathcal{X}$ for the flag complex of admissible dispositions (Section 2.2):

$$(\mathbf{T}\text{-Intro}_\kappa) \quad \frac{\Gamma; \Delta \vdash e : T \quad \text{kinds}(\Delta) \cup \{\kappa\} \in \mathcal{X} \quad \kappa_0 \notin \text{caps}(f)}{\Gamma; \Delta, l : \text{Witness}(T) \vdash \text{intro}_\kappa(e) : \text{Witness}(T)} \quad (l \text{ fresh})$$

The rule covers the obligations tacet can introduce — Reify and Bound directly, and Sync through the effectful primitives; Break has no introduction rule and enters only through the contract. The admissibility premise is what makes the “impossible combinations” of Section 2.2 a typing fact rather than prose: an introduction that would leave the flag complex simply has no derivation. The capability premise $\kappa_0 \notin \text{caps}(f)$ selects the *live* case for the renounceable kinds Unbounded and Awaited; the renounced case is the following rule.

When the corresponding capability is declared, the same introduction form is *transparent* to the linear context: it adds no obligation to Δ , recording instead a null-obligation witness in W_0 — bookkeeping that owes nothing and ultimately maps to Total (Section 2.2):

$$(\mathbf{T}\text{-}\mathbf{Absorb}_\kappa) \quad \frac{\Gamma; \Delta \vdash e : T \quad \kappa_0 \in \text{caps}(f)}{\Gamma; \Delta \vdash \text{intro}_\kappa(e) : T'} \quad \kappa \in \{\text{Unbounded}, \text{Awaited}\}.$$

The result type T' is T for a renounced recursion (Unbounded_0 in the contract), which yields the value it would have computed, and the trivial type for a renounced asynchronous launch (Awaited_0), which returns immediately as a fire-and-forget sink. The linear context is unchanged: a W_0 witness exposes no obligation, so renunciation adds nothing to Δ . Because asynchrony enters only through primitive effectful functions, Awaited_0 renounces the Sync obligation those primitives would otherwise raise.

The witness contract

Where *tacet* introduces *obligations* in the body, a function's contract declares the *witnesses* it exposes in its interface. The two are the syntactic faces of the thick/thin distinction of Section 2.3: obligations are the thin occurrences that live in Δ ; witnesses are the thick objects the signature advertises. A declaration may append a symbol-construct after the return type,

$$f : A \rightarrow \tau \ \kappa_1 \cdots \kappa_n,$$

naming the witnesses the body exposes. This contract is the function's set of capabilities, and $\text{caps}(f)$ is the subset of renounced symbols within it — those written κ_0 , a witness with its obligation removed. The contract must *comply with the body*: the witnesses it declares must match the obligations the branches raise, up to renunciation. Left empty, it is inferred — the function is linearly typed by the evaluation of its branches. The intent is to be *always explicit about obligations*, which the body's *tacets* make unavoidable, while *opting in to witnesses*, which the contract makes a deliberate part of the interface, since the witness API is itself something the programmer defines.

Loop is the one witness that can *only* be declared, never introduced. Because its Break cannot be raised in code, a loop reaches the type system solely by adding Loop to the contract, which must be made explicit and must comply with the function's main branch. This matches the isolation of Loop in the flag complex (Section 2.2): a loop stands alone as a witness, separated from every other, so a Loop contract admits no companion. Renunciation is declared the same way — Unbounded_0 or Awaited_0 in the contract absorbs the corresponding obligation into Total (Section 2.2), which is how a programmer opts into possible unboundedness or a fire-and-forget effect.

The correspondence $L_f \leftrightarrow \Delta$

The semantic collection of Section 2.3 and the syntactic context here are two views of one object. For a branch $b \in B_f$, the introduction rules accumulate exactly the occurrences of obligations(b) as linear hypotheses; write Δ_b for that

context. The discharge rules of Section 2.5 transform Δ_b . A branch is **complete** when it is typeable with an empty residual context,

$$\Gamma; \cdot \vdash \text{body}_b : \tau,$$

i.e. every occurrence in $\text{obligations}(b)$ has been discharged. Collecting over branches recovers

$$L_f = \coprod_{b \in B_f} \text{obligations}(b)$$

as the total space of occurrences the type system must drive to empty. **Global Branch Totality** is precisely the requirement that every branch of a well-typed function admit such a derivation: no branch completes into V while an obligation remains, and — by the fuel discipline of Section 2.1 — no branch defers indefinitely. The introduction rules populate Δ ; the discharge rules drain it; a well-typed branch is one for which the draining is complete.

2.5 Discharge

Introduction populates the linear context; discharge drains it. This section gives the interface through which every obligation leaves Δ — a single family of three reifiers, uniform across all witness kinds — and the rules that combine subterms and branches.

The two contexts

Judgments have the form $\Gamma; \Delta \vdash e : \tau$ with two environments of different character:

- Γ , the **lexical context**, maps immutable identifiers to ordinary value types. It is unrestricted: weakening and contraction are admissible.
- Δ , the **linear context**, is a multiset of active, unconsumed obligation occurrences, $\Delta = \{l_1 : \text{Witness}(T_1), l_2 : \text{Witness}(T_2), \dots\}$. It admits neither weakening nor contraction: an occurrence can be neither discarded nor duplicated.

Sequential subterms — subterms both of which are evaluated — split the linear context **multiplicatively**, $\Delta = \Delta_1, \Delta_2$, distributing the active obligations disjointly between them. Alternatives combine additively, as the branch rule below records.

The three reifiers

Whatever obligation a witness exposes, it is discharged by one of three reifiers, distinguished only by *where the result lands*:

$$\text{reify}_T : W \rightarrow V(T) \quad \text{reify}_V : W \rightarrow V(U) \quad \text{reify}_L : W \rightarrow W_L.$$

All three consume a witness. reify_T and reify_V reify it into the value space — into a value of the witness's own type, or of an arbitrary target type. reify_L reifies

it into the linear space instead, producing a fresh obligation. There is no per-kind discharge machinery: a single interface answers partiality of every kind, which is the point of collapsing the witness taxonomy onto one reification API.

$$(\mathbf{T}\text{-Reify}_T) \quad \frac{\Gamma; \Delta_1 \vdash e_w : \text{Witness}(T) \quad \Gamma, x:T; \Delta_2 \vdash e_h : T}{\Gamma; \Delta_1, \Delta_2 \vdash \text{reify}_T(e_w, x.e_h) : T}$$

$$(\mathbf{T}\text{-Reify}_V) \quad \frac{\Gamma; \Delta_1 \vdash e_w : \text{Witness}(T) \quad \Gamma, x:T; \Delta_2 \vdash e_h : U}{\Gamma; \Delta_1, \Delta_2 \vdash \text{reify}_V(e_w, x.e_h) : U}$$

$$(\mathbf{T}\text{-Reify}_L) \quad \frac{\Gamma; \Delta_1 \vdash e_w : \text{Witness}(T) \quad \Gamma, x:T; \Delta_2 \vdash e_d : \text{Witness}(U)}{\Gamma; \Delta_1, \Delta_2 \vdash \text{reify}_L(e_w, x.e_d) : \text{Witness}(U)}$$

In each rule the witness e_w carries an obligation $l \in \Delta_1$, and the handler consumes the payload variable $x : T$. In $\mathbf{T}\text{-Reify}_T$ and $\mathbf{T}\text{-Reify}_V$ the obligation is discharged into a value and no residue remains. In $\mathbf{T}\text{-Reify}_L$ the obligation is consumed and a fresh successor $l' : \text{Witness}(U)$ is threaded into the context; the successor is always *Partial*, so deferral, whatever kind of witness it consumes, hands forward an ordinary reification obligation.

The type discipline of reification

reify_T and reify_V differ not operationally but in the discipline they commit to, and the difference is deliberately visible in the syntax. reify_T is **type-preserving** (endomorphlic): it obliges the handler to return a value in the witness's own type, so reification lands back in the domain the computation was expected to inhabit. reify_V is **type-transforming** (polymorphic): it permits projection into an arbitrary target U , allowing a branch to leave that domain. Although reify_T coincides with reify_V at $U := T$, the two are kept distinct precisely because their distinctness is *statically legible*.

Call a branch **codomain-coherent** when all of its reifications are type-preserving, and **codomain-divergent** when it uses at least one type-transforming reification. This predicate is decidable by inspection of the discharge sites, and it measures how tightly a branch stays within the domain fixed by its structure. It feeds signature honesty directly: a function whose branches are all codomain-coherent has honest codomain τ , whereas each reify_V site widens the honest codomain to include its escape type — and because the widening is marked by a distinct construct, the true codomain is statically recoverable rather than inferred. Type-preserving reification is the default that keeps a function within one declared codomain; type-transforming reification is the explicit, analyzable act of leaving it.

Deferral chains

Because reify_L yields a fresh obligation, obligations may be threaded through successive deferrals before a value is produced:

$$l^{(0)} \xrightarrow{\text{reify}_L} l^{(1)} \xrightarrow{\text{reify}_L} \dots \xrightarrow{\text{reify}_L} l^{(k)} \xrightarrow{\text{reify}_T|_V} v \in V.$$

Two independent properties govern such a chain. **Local linearity** is a property of Δ : each reify_L step consumes its obligation $l^{(i)}$ exactly once, producing the successor $l^{(i+1)}$ — no occurrence is duplicated or dropped. **Chain finiteness** is a property of the fuel discipline: each reify_L draws one unit of fuel (Section 2.1), so in the default regime the length k is bounded and the chain is finite. A finite chain terminates either by reaching a value-producing reifier, or by exhausting fuel — in which case the residue is itself an Unbounded witness, handled by the same discipline. The two properties are orthogonal — linearity is enforced by the context, finiteness by the fuel — and together they establish that no branch defers forever and none completes with an obligation outstanding.

Additive branch combination

The multiplicative split governs subterms that both execute. Alternatives are different: since select_f executes exactly one branch, the branches *share* the incoming linear context and are each typed against it independently:

$$\text{(T-Branch)} \quad \frac{\Gamma; \Delta \vdash \text{body}_b : \tau \quad \text{for every } b \in B_f}{\Gamma; \Delta \vdash \text{select}_f : \tau}$$

Each branch introduces and discharges its own occurrences internally, so no occurrence from one branch is ever live during another. The collected space $L_f = \coprod_b \text{obligations}(b)$ is thus a *disjunction* of admissible faces — one per branch — never their union. This is what forbids a function whose outcome space conflates disciplines the composition rules keep apart. Under multiplicative combination a function with an Unbounded branch and an Awaited branch would accumulate the inadmissible face {Unbounded, Awaited} and fail to typecheck, though no single execution realizes both; additivity records each branch’s disposition as an independent alternative, so a function may *span* several maximal profiles across its branches while every individual outcome remains a single admissible face. Domain segregation is enforced at the granularity of branches, not smeared across them.

Summary of linear-context transformations

$$\text{T-Reify}_T : \Delta, l : \text{Witness}(T) \longrightarrow \Delta$$

$$\text{T-Reify}_V : \Delta, l : \text{Witness}(T) \longrightarrow \Delta$$

$$\text{T-Reify}_L : \Delta, l : \text{Witness}(T) \longrightarrow \Delta, l' : \text{Witness}(U) \quad (l' \text{ Partial})$$

Every obligation $l \in L_f$ is therefore either discharged into the value space V or transformed into a successor obligation l' ; none can be silently dropped. Together with the introduction rules of Section 2.4, these judgments establish the invariant on which the whole calculus rests: a well-typed branch drives its linear context to empty.

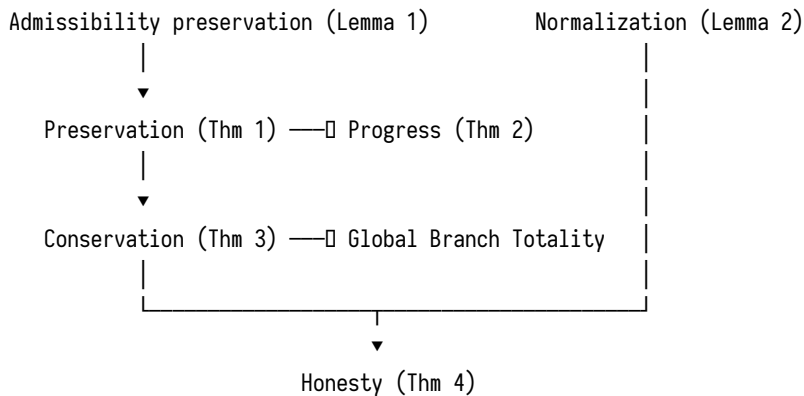
This invariant is asserted here but not yet proved. Establishing it — and, through it, the honesty that the calculus was built to guarantee — is the work of the next chapter, which develops the metatheory of the system defined in Sections 2.1–2.5.

3 Metatheory

Chapter 2 defined the calculus and closed with an invariant it did not prove: that a well-typed branch drives its linear context to empty, so that no obligation is ever silently dropped. This chapter establishes that invariant and, through it, the property the calculus was designed to guarantee — that a function’s signature tells the whole truth about what its evaluation can do. Statements are given in full and with full rigor; proofs are sketched, with the remaining details deferred to a companion development. The sketches are written to be read, not merely certified: each is meant to make clear *why* the result holds, so that the honesty theorem is understood before its proof is consulted.

3.1 What we prove, and why it matters

The results of this chapter are not independent; they form a short dependency chain, and seeing the chain first makes the individual statements legible.



Two structural lemmas underwrite everything. *Admissibility preservation* says reduction never leaves the algebra of admissible dispositions; *normalization* says the fuel-bounded, loop-free fragment always terminates. On the first rest *preservation* and *progress* — the usual syntactic-soundness pair, guaranteeing that types are stable under reduction and that a well-typed program is never stuck. Preservation in turn yields *conservation*: a value carries no outstanding obligation, so every obligation raised is answered exactly once. Conservation is Global Branch

Totality restated, and together with normalization it delivers the capstone, *honesty*.

Informally, honesty is the following. A mainstream signature $f : A \rightarrow \tau$ oversells: evaluation may also throw or diverge, invisibly. In Tacet, once a function is well-typed, its signature and the dispositions of its branches disclose *every* way evaluation can depart from returning a value of type τ — failure, deferral, unbounded recursion, productive looping — and nothing is left to a silent exception, a hidden default, or a bottom value. Honesty is disclosure, not the elimination of everything that could go wrong.

3.2 The formal system, recalled

For self-containment we recall the notation of Chapter 2; nothing here is new.

Configurations and fuel. Reduction acts on configurations $\langle e, \varphi \rangle$ with a fuel value $\varphi \in \mathbb{N} \cup \{\infty\}$. Exactly two reduction forms draw one unit of fuel — the unfolding of a recursive call and the deferral reify_L — and at $\varphi = 0$ such a redex introduces a Bound obligation (typed as an Unbounded witness) rather than becoming stuck. Every other step leaves φ fixed. An intrinsically-infinite Loop does *not* draw fuel. A function evaluates with $\varphi = \infty$ exactly when it declares the Unbounded_0 capability, recorded in $\text{caps}(f)$.

The result domain. Branch evaluation lands in $R = V \uplus W$ with $W = W_0 \uplus W_L$: V is the value domain (Total outcomes), W_L the witnesses that expose a linear obligation, and W_0 the null-obligation witnesses — renounced bookkeeping that owes nothing and ultimately maps to Total. Bottom is never an element: $\perp \notin R$, unconditionally.

The two contexts. Judgments $\Gamma; \Delta \vdash e : \tau$ carry an unrestricted lexical context Γ (weakening and contraction admissible) and a linear context Δ — a multiset of unconsumed obligation occurrences, admitting neither. Sequential subterms split Δ multiplicatively; alternative branches share it additively.

Dispositions. The live witness kinds of a context, $\text{kinds}(\Delta) \subseteq \mathcal{K}_L = \{\text{Partial}, \text{Unbounded}, \text{Awaited}, \text{Loop}\}$, must at all times form a face of the flag complex $\mathcal{X}$, whose maximal faces are $\{\text{Partial}, \text{Unbounded}\}$, $\{\text{Partial}, \text{Awaited}\}$, and $\{\text{Loop}\}$.

The rules. Introduction (**T-Intro** $_{\kappa}$, guarded by admissibility; **T-Absorb** $_{\kappa}$ for the renounced case) deposits occurrences into Δ ; the three reifiers reify_T , reify_V , reify_L discharge them — the first two into V , the third into a fresh Partial successor; **T-Branch** combines alternatives additively.

3.3 Structural lemmas

Lemma 1 (Admissibility preservation). *If $\Gamma; \Delta \vdash e : \tau$ with $\text{kinds}(\Delta) \in \mathcal{X}$, and $\langle e, \varphi \rangle \rightarrow \langle e', \varphi' \rangle$, then $\Gamma; \Delta' \vdash e' : \tau$ with $\text{kinds}(\Delta') \in \mathcal{X}$.*

Proof sketch. The live kinds change only at introduction and discharge. Introduction is guarded: **T-Intro** $_{\kappa}$ fires only when $\text{kinds}(\Delta) \cup \{\kappa\}$ is a face, so it cannot leave $\mathcal{X}$. Type-preserving and type-transforming reification remove an occurrence, and a face minus a vertex is again a face, since $\mathcal{X}$ is downward closed. Deferral replaces an occurrence by a Partial successor; Partial is compatible with every face except those containing Loop, and Loop never co-occurs with a live obligation in the first place (its introduction demands an empty Δ). Every reachable context is therefore admissible. ■

The lemma is what turns Chapter 2’s “impossible combinations” from prose into an operational fact: the type system cannot even transiently reach a disposition outside the complex.

Lemma 2 (Normalization of the bounded, loop-free fragment). *If $\cdot; \cdot \vdash e : \tau$ with $\text{caps}(e) = \emptyset$ and e contains no Loop, then every reduction sequence from $\langle e, \varphi \rangle$, for any $\varphi \in \mathbb{N}$, is finite.*

Proof sketch. Assign to each configuration the pair $(\varphi, \|e\|)$, where $\|e\|$ is the structural size of e , and compare pairs lexicographically. The two fuel-drawing steps — recursion and reify_L — strictly decrease the first component. Every other reduction leaves the first component untouched and strictly decreases the second, since it removes a redex without introducing unbounded regeneration. A strictly descending sequence in $\mathbb{N} \times \mathbb{N}$ under the lexicographic order cannot be infinite, because that order is well-founded. Hence no infinite reduction exists.

The measure also explains precisely which programs are excluded. The first component can fail to decrease only when $\varphi = \infty$, which is the Unbounded_0 capability; the second can fail to decrease only when a construct regenerates itself without consuming fuel, which is exactly an intrinsic Loop. These are the two — and only two — sources of legitimate non-termination in the calculus, and both are disclosed. ■

3.4 Syntactic soundness

Theorem 1 (Preservation). *If $\Gamma; \Delta \vdash e : \tau$ and $\langle e, \varphi \rangle \rightarrow \langle e', \varphi' \rangle$, then $\Gamma; \Delta' \vdash e' : \tau$, where Δ' is related to Δ by exactly one of*

$$\begin{aligned} \text{intro:} \quad & \Delta' = \Delta, l : \text{Witness}(T) & (\text{kinds}(\Delta) \cup \{\kappa\} \in \mathcal{X}) \\ \text{reify}_T, \text{reify}_V : & \Delta' = \Delta \setminus \{l\} \\ \text{reify}_L : & \Delta' = (\Delta \setminus \{l\}), l' : \text{Witness}(U) \quad (l' \text{ Partial}) \\ \text{other:} \quad & \Delta' = \Delta, \end{aligned}$$

and $\varphi' \leq \varphi$ in every case, with $\varphi' < \varphi$ on recursion and reify_L .

Proof sketch. Induction on the typing derivation, using the substitution lemmas appropriate to each context — weakening-and-contraction substitution for Γ , multiplicative (occurrence-splitting) substitution for Δ . Each reduction rule of the operational semantics corresponds to one typing rule of Sections 2.4–2.5; matching them gives the type τ unchanged and the stated transformation of Δ , read directly off the rule. The fuel inequality is immediate from the reduction relation, in which only recursion and reify_L decrement φ . ■

The theorem records the one fact the rest of the chapter leans on: reduction changes Δ only in the four disciplined ways listed, so the linear context is never altered behind the type system’s back.

Theorem 2 (Progress). *If $\cdot; \Delta \vdash e : \tau$, then either e is a value in $V(\tau)$, or $\langle e, \varphi \rangle$ takes a step.*

Proof sketch. By canonical forms on the type of the leftmost redex. The two situations that might otherwise strand a computation both fail to arise. Fuel exhaustion is not a stuck state: a fuel-drawing redex at $\varphi = 0$ steps to a Bound obligation. And a reifier cannot be misapplied: it is typed to demand an occurrence of W_L witness type $\text{Witness}(T)$, so it never meets an obligation-free target — a

Total value, or a null-obligation W_0 witness — on which discharge could fail. A Loop, finally, is always able to iterate, and may additionally receive an out-of-band break. ■

3.5 Linear soundness

Theorem 3 (Conservation). *If $\cdot; \Delta \vdash v : \tau$ and v is a value, then $\Delta = \cdot$. Consequently every obligation raised along a reduction is either discharged into V or deferred to a unique Partial successor — exactly once, never duplicated, never dropped.*

Proof sketch. A value inhabits V , and V is the Total fibre of the result domain, whose disposition is empty. So a value has no live obligation, i.e. $\Delta = \cdot$. That every obligation is consumed exactly once is then the content of the linear-context transformation summary of Section 2.5, read cumulatively: introduction adds an occurrence, the two value-reifiers remove one, deferral swaps one for a successor, and linearity forbids any rule that would silently discard a live occurrence. ■

Corollary (Global Branch Totality). *Every complete branch of a well-typed function is typeable as $\Gamma; \cdot \vdash \text{body}_b : \tau$: it reaches a value only with an empty linear context. The “no value” outcome — the loud, terminal failure of Section 2.3 — is therefore unreachable in well-typed code.*

This corollary is the precise form of the invariant Chapter 2 asserted. A branch cannot slip into the value domain while owing an obligation, and it cannot defer forever (Lemma 2); so its only route to completion is to answer every obligation it raised.

3.6 Honesty

Everything above converges here.

Theorem 4 (Honesty). *Let $\cdot; \cdot \vdash e : \tau$.*

(i) *Bounded honesty. If $\text{caps}(e) = \emptyset$ and e contains no Loop, then evaluation terminates and $\llbracket e \rrbracket \in V(\tau)$: a value of exactly the declared type. No witness escapes, by Conservation; no exceptional outcome escapes, since E is reified as Partial obligations and discharged; and no divergence occurs, by Lemma 2. The gap opened in Chapter 1 closes entirely,*

$$\llbracket f \rrbracket : A \rightarrow \tau + E + \{\perp\} \quad \text{reduces to} \quad f : A \rightarrow \tau.$$

(ii) *General honesty. Without those two restrictions, the outcome of an intermediate computation lies in*

$$V \uplus (W_0 \uplus W_L), \quad \perp \notin R \text{ unconditionally,}$$

*and every non-value outcome is disclosed: exceptional termination is reified into W_L as Partial obligations, and non-termination arises only under a declared Unbounded₀ capability — visible in the signature — or within a Loop branch — visible in that branch’s disposition. The **top-level** computation — the one wrapping the whole stack — must land in*

$$V \uplus W_0 :$$

every W_L obligation has been discharged by that point, and what may remain is only null-obligation bookkeeping in W_0 , which owes nothing and maps to Total. In no case is a bottom value produced; $\perp$ appears nowhere in the outcome space, and no obligation escapes undischarged.

Proof sketch. Part (i) combines Conservation (no obligation and no exceptional outcome escapes a value) with Lemma 2 (the fragment terminates), leaving $V(\tau)$ as the only possible result. Part (ii): Preservation and Lemma 1 confine every reduction to the admissible fragment, so the sole steps beyond the terminating argument of (i) are those unlocked by a declared capability — which record a null-obligation W_0 witness — and Loop iteration, whose non-termination is carried by a Break obligation. Each such source is recorded in $\text{caps}(e)$ or in a branch's disposition, both parts of the signature. Conservation drives every W_L obligation to discharge before the outermost computation completes, leaving its result in $V \uplus W_0$; and since $\perp \notin R$ holds unconditionally, no reduction ever yields a bottom value. ■

A worked trace

To see the machinery discharge a single obligation end to end, take a partial head function on integer lists. Its empty branch has no value to return, so it raises an obligation with tacet :

$$\text{head}(xs) = \text{select} \begin{cases} xs = [] & \mapsto \text{tacet}(\text{Empty}) \\ xs = h :: _ & \mapsto h \end{cases}$$

The non-empty branch is Total: it returns $h : \text{Int}$ with $\Delta = \cdot$. The empty branch is not. Evaluating $\text{tacet}(\text{Empty})$ introduces a Reify obligation of payload type Int into Δ — the symbol Empty is its value-level proxy — and forces a Partial witness into head 's contract; the branch now carries $\Delta = \{l : \text{Witness}(\text{Int})\}$, where l is the obligation and $\text{Witness}(\text{Int})$ records the payload type inherited from that forced witness. By Global Branch Totality the branch cannot complete in this state, so the obligation must leave. It has three exits, and the honest signature of head is determined by which is taken.

If head discharges nothing itself, the obligation surfaces at the boundary as its forced witness, and head is honestly typed

$$\text{head} : \text{List Int} \rightarrow \text{Witness}(\text{Int}),$$

announcing to every caller that the result must be reified before it can be used as an Int . When a caller invokes head , that witness exposes the obligation into the caller's own Δ (Section 2.3); a caller that wants a fallback then discharges it with a type-preserving reifier, and the linear context empties:

$$\underbrace{\{l : \text{Witness}(\text{Int})\}}_{\text{obligation live}} \xrightarrow{\text{reify}_T(\cdot, x. 0)} \text{discharged}, \quad \text{result} \in V(\text{Int}).$$

Had the caller instead deferred first — $\text{reify}_L(\cdot, x. \text{tacet}(\dots))$ — the occurrence would have been replaced by a fresh Partial successor l' , threading the obligation one step downstream, each step drawing fuel, until some later reify_T or reify_V landed it in V . At no point is the empty case allowed to masquerade as an Int ,

and at no point does a bottom appear. The signature of head is exactly as honest as its code: `Int` if the failure is answered internally, `Witness(Int)` if it is passed on.

$$\underbrace{\{l : \text{Witness}(\text{Int})\}}_{\text{obligation live}} \xrightarrow{\text{reify}_T(\cdot, x.0)} \text{ب}, \quad \text{result} \in V(\text{Int}).$$

discharged

Had the caller instead deferred first — $\text{reify}_L(\cdot, x. \text{tacet}(\dots))$ — the occurrence would have been replaced by a fresh `Partial` successor l' , threading the obligation one step downstream, each step drawing fuel, until some later reify_T or reify_V landed it in V . At no point is the empty case allowed to masquerade as an `Int`, and at no point does a bottom appear. The signature of head is exactly as honest as its code: `Int` if the failure is answered internally, `Witness(Int)` if it is passed on.

What honesty does not claim

The theorem is deliberately modest in three respects, and stating the limits sharpens the claim.

It does not decide the halting problem. Honesty never asserts that an `Unbounded0` function terminates; it asserts only that the *possibility* of non-termination is declared rather than hidden. The calculus trades a guarantee it cannot give for a disclosure it can.

It does not eliminate failure, divergence, or effects. A `Partial` function may still fail, a `Loop` may still run forever, an `Awaited0` effect may still fire and be forgotten. What honesty removes is not these phenomena but their *invisibility*: each is forced to appear in a type, a capability, or a disposition.

And it does not make every well-typed program total. Totality in the strong sense holds only for the fragment of part (i). Elsewhere, non-termination remains possible — but only along the two disclosed routes, and never as a silent $\perp$. This is the exact sense in which `Tacet` is honest rather than merely safe: it does not promise that nothing goes wrong, only that nothing goes wrong *unannounced*.